TechWithKoushik

THE COMPLETE TECHNICAL MIGRATION PLAYBOOK

Software Engineer → AI Engineer

From Software Engineer to AI Engineer in 2026

Don't rewrite your career. **Migrate** it. A dense, technical, no-fluff field guide — skills, code, architectures, a 90-day plan, portfolio projects, and interview prep — built to take you from shipping software to shipping AI.

3.2:1	70%	90	$176K+
Open roles vs qualified people	Skills you already own	Days to interview-ready	Median AI engineer salary

A free companion guide to the video.
Watch the full breakdown on YouTube — **TechWithKoushik**.

GET MORE FREE RESOURCES
koushikmaji.com

READ THIS FIRST

What's Inside & How to Use It

This is not a motivational listicle. It is a **working reference** you can keep open in a second monitor while you build. Every section is designed so that when you finish reading it, you can immediately go do the thing. Skim the maps, then live in the code.

Part I — The Landscape & Mindset

- 01 · The 2026 market & the three AI tracks
- 02 · AI Engineer vs ML Engineer vs Data Scientist
- 03 · The "Migrate, don't rewrite" model & the 70/30 rule
- 04 · The skill pyramid — what to learn, what to skip

Part II — The Core Technical Stack

- 05 · LLM fundamentals & prompt engineering (code)
- 06 · Tool / function calling & the agent loop (code)
- 07 · RAG deep dive: chunking, embeddings, hybrid, rerank
- 08 · Vector databases compared + a real pipeline (code)
- 09 · Evaluation: the skill that gets you hired (RAGAS)

Part III — Agents, MCP & Production

- 10 · AI agents & frameworks (LangGraph, CrewAI...)
- 11 · Model Context Protocol (MCP): the USB-C for AI
- 12 · Shipping to production: FastAPI + Docker (code)
- 13 · Cost & latency optimization playbook
- 14 · Guardrails & safety (prompt injection, PII)

Part IV — Getting Hired

- 15 · The 90-day migration plan (30/30/30)
- 16 · Portfolio projects that actually get you hired
- 17 · Interview prep: system design + real Q&A
- 18 · Salary, the closing window & your 7-day kit

THE ONE THING TO REMEMBER

You are not starting over. You are an experienced system adding one new layer. Most of what makes an AI engineer valuable — shipping discipline, APIs, testing, reliability instincts — you already have. This guide is about the **30% that's new**, taught the way an engineer actually wants to learn it: concrete, with code.

01. The 2026 Market & the Three AI Tracks

"AI Engineer" barely existed as a title in 2022. By 2026 it is the fastest-growing job title in the US for two years running, with demand outstripping qualified supply by roughly **3.2 to 1** and posting growth near **143% year-over-year**. The bottleneck is not open roles — it is qualified people. That gap is your opportunity.

~1.6M	~518K	+143%	#1
Open AI engineer roles	Qualified candidates	YoY posting growth	Fastest-growing US title

"AI Engineer" is really three different jobs

The single biggest source of confusion — and wasted study time — is treating AI engineering as one role. It is at least three, with very different entry bars. As a software engineer, **Track 2 is your lane.**

Track	What you do day-to-day	Entry bar	Fit for SWE?
1. ML / Research Engineer	Train models, implement papers, run training loops in PyTorch/JAX, alignment research	PhD often expected; heavy math	Hard pivot
2. AI (Application) Engineer	Build products on foundation models: RAG, agents, prompting, evals, deployment	Strong SWE + LLM API fluency	**Your lane — ~80% of jobs**
3. MLOps / AI Infra Engineer	Model serving, GPU clusters, monitoring, pipelines, reliability at scale	DevOps + cloud + K8s	Great if you're infra-minded

WHY THIS IS GOOD NEWS FOR YOU

The years of breakthrough research needed to build a capable LLM are **already done**. GPT, Claude, Gemini and open models exist and are one API call away. The value has moved **up the stack** to the people who can reliably build products on top of them — which is exactly what software engineers are trained to do.

What an AI Engineer actually does on a Tuesday

- Debug why a RAG pipeline is returning irrelevant chunks and fix the chunking/retrieval.
- A/B test two system prompts and measure which produces more accurate, grounded output.
- Integrate a new model API into an existing service behind a clean interface.
- Write an evaluation harness that automatically grades 500 model responses for faithfulness.
- Add caching + routing to cut the LLM bill by 60% without hurting quality.

Notice: that is backend engineering with a probabilistic core. Not research.

02. The Roles, and Why You're 70% There

Dimension	Data Scientist	ML Engineer	AI Engineer (you)
Primary focus	Insight & analysis	Training & serving models	LLM-powered product features
Core skills	Stats, SQL, viz	Pipelines, MLOps, optimization	Prompting, RAG, agents, LLM APIs
Tools	Jupyter, pandas	PyTorch, MLflow, Kubeflow	LangChain/LlamaIndex, vector DBs, SDKs
Output	Dashboards, models	Trained models, serving infra	Deployed AI features & pipelines
Math needed	Moderate	Heavy	**Light** (embeddings, basic stats)

03. Migrate, Don't Rewrite

When you upgrade a large production system, you don't demolish it and start from zero — you add a new layer on top of what already works. Your career is the same. Becoming an AI engineer is a **migration**, not a rewrite. Here is the honest accounting of what transfers.

The 70% you already own

- **REST APIs** → every model is an API call
- **Git / versioning** → versioned prompts & pipelines
- **System design** → RAG & agent architecture
- **Testing / debugging** → evaluation & observability
- **Async / concurrency** → parallel tool & API calls
- **"What breaks at 3am?"** → reliability = your moat

The 30% that's new (this guide)

- LLM APIs & the request/response mental model
- Prompt engineering as interface design
- RAG: retrieval, chunking, embeddings, rerank
- Agents & tool calling (ReAct loop, MCP)
- Evaluation of non-deterministic systems
- Cost, latency & guardrails in production

WHAT TO UN-LEARN

Traditional code is deterministic: same input, same output. LLMs think in **probabilities**. The same prompt can vary, there is rarely one "correct" output (only better/worse), and you can't "ship and forget" — you must keep evaluating. Internalize this shift and everything else clicks.

04. The Skill Pyramid — and the Tier Trap

The most expensive mistake transitioning engineers make: pouring months into **Tier 3** (fine-tuning, heavy math, training from scratch) while skipping **Tier 2** — where roughly 80% of the actual jobs live. You end up studying for the wrong exam. Master the middle.

TIER 3 · Nice-to-have (defer)

Fine-tuning & LoRA/QLoRA · PyTorch training loops · quantization & serving internals · deep linear algebra. **Useful later — rarely the thing that gets you hired.**

TIER 2 · High-signal (LIVE HERE) — the golden zone

This is what interviews test and what ships products:

LLM APIs · Prompt engineering · RAG · Vector databases · Evaluation / evals · AI agents · Tool / function calling · MCP · Structured outputs

TIER 1 · Foundations (you have these)

Python (async, type hints, pydantic) · REST APIs & HTTP · SQL · Git · Docker basics · a cloud platform. **Your existing SWE experience already covers most of this.**

DO FIRST

Get fluent calling LLM APIs, build a real RAG system, and learn to evaluate it. That trio alone makes you hireable for the majority of open AI-engineer roles.

DO LATER

Fine-tuning, custom training, GPU/quantization deep-dives. Come back once you're hired and getting **paid** to learn them — or if you deliberately target research/infra tracks.

The hiring team's real checklist (2026)

- **Engineering fundamentals:** clean Python, API design, Git, tests that catch regressions.
- **LLM app skills:** prompt design with intent, structured outputs, tool calling, predictable failure handling.
- **RAG skills:** embedding selection, chunking, retrieval logic, reranking, grounded/cited answers.
- **Evaluation mindset:** quality checks, groundedness scoring, versioned evals, regression tests.
- **Deployment basics:** containerized services, monitoring, and clear cost/latency/reliability trade-offs.
- **Safety awareness:** access control, PII discipline, guardrails, auditability.

05. LLM Fundamentals & Prompt Engineering

Everything starts with one primitive: a call to a model. Treat the LLM like an **unreliable external service** — wrap it, validate its output, handle failures, and never trust it blindly. Here is the minimum viable mental model plus the first call.

```
# The atomic unit of AI engineering: a typed LLM call
from openai import OpenAI
client = OpenAI()

def ask(user_msg: str) -> str:
    resp = client.chat.completions.create(
        model="gpt-4o-mini",          # cheap tier for simple tasks
        temperature=0.2,                # low = factual / consistent
        messages=[{"role":"system","content":"You are concise."},
                  {"role":"user",  "content": user_msg}])
    return resp.choices[0].message.content
```

The prompt patterns that actually move the needle

Pattern	What it does	When to reach for it
Role + Context + Task + Format	The reliable base structure for any prompt	Always — your default template
Few-shot	2–5 input/output examples; +10–40% on format-strict tasks	Classification, extraction, tone
Chain-of-Thought	"Think step by step" → big lift on reasoning/math	Multi-step logic (many models do this internally now)
Structured output	Force JSON/schema-valid responses	Anytime downstream code parses the output
ReAct	Thought → Action → Observation loop	Agents that use tools (Section 06 & 10)
Prompt chaining	Decompose a big task into small, testable steps	Complex workflows; easier to eval & debug

Structured output = the pattern juniors skip

Production systems parse model output with code, so you need **typed, schema-valid** responses, not prose. Pair a Pydantic schema with the provider's structured-output mode:

```
from pydantic import BaseModel

class Ticket(BaseModel):
    severity: str        # "low" | "medium" | "critical"
    category: str
    summary:  str

resp = client.beta.chat.completions.parse(
    model="gpt-4o-mini", response_format=Ticket,   # schema-valid JSON
    messages=[{"role":"user","content": "Users cannot log in after deploy."}])
ticket: Ticket = resp.choices[0].message.parsed
```

Engineer's rule: *version prompts like code, and never let raw model text flow straight into business logic.*

06. Tool / Function Calling & the Agent Loop

Tool calling is what turns a text generator into something that can **act** — query a database, hit an API, run a calculation. The critical mental model: **the LLM is a router, not an executor.** It only **decides** which function to call and with what arguments. **Your code runs it.** The model never touches your keys, DB, or filesystem — that separation is both the security boundary and the pattern.

The universal 5-step loop (works across OpenAI, Anthropic, Gemini)

1. **Define** tools as JSON-schema definitions (name, description, typed params).
2. **Send** the user message + tool definitions to the model.
3. **Detect** a tool call in the response (model returns structured args).
4. **Execute** the function in **your** code and capture the result.
5. **Return** the result to the model, which reasons over it and answers.

```
# One tool, the full loop
tools = [{
  "type": "function",
  "function": {
    "name": "get_order_status",
    "description": "Look up the delivery status for an order id.",
    "parameters": {"type":"object",
      "properties":{"order_id":{"type":"string"},
      "required":["order_id"]}}}]

msgs = [{"role":"user","content":"Where is order A123?"}]
r = client.chat.completions.create(model="gpt-4o", messages=msgs, tools=tools)

call = r.choices[0].message.tool_calls[0]      # model REQUESTS the call
args = json.loads(call.function.arguments)
result = get_order_status(**args)                 # YOUR code executes it

msgs += [r.choices[0].message,
         {"role":"tool","tool_call_id":call.id,"content":result}]
final = client.chat.completions.create(model="gpt-4o", messages=msgs)  # grounded answer
```

Reliability best-practices (interviewers probe these)

- **Keep the toolset small & scoped.** 50 tools when 5 will do measurably degrades selection accuracy and burns tokens (each definition adds ~100–300 input tokens). Load tools dynamically when the catalog grows.
- **Argument correctness > tool selection.** A polished, well-described schema beats a clever prompt — most failures are wrong **arguments**, not the wrong tool.
- **Parallelize independent calls.** Parallel tool execution can cut end-to-end latency dramatically vs sequential.
- **Return structured errors** so the model can self-correct instead of retrying from scratch.
- **Gate write/consequential actions** behind validation or a human-in-the-loop approval.

TRUST BOUNDARY FIRST

Before you list tools, define what the agent may do **without** confirmation. Separate reads from writes, validate every argument, use idempotency keys, and keep an audit trail. This single instinct separates strong candidates from weak ones in the agents interview round.

07. RAG Deep Dive: the Most-Hired Skill

Retrieval-Augmented Generation grounds an LLM in **your** data at query time — the default architecture for enterprise AI. Here is the reality check: naive RAG fails at retrieval roughly **40–73% of the time** in production. When RAG breaks, it is the **retrieval** step, not generation. Master retrieval and you are ahead of most candidates.

Two pipelines: indexing (offline) and query (online)

Indexing pipeline (offline)

1. **Ingest** PDFs/docs/wikis; extract text + metadata
2. **Clean** boilerplate; preserve headings/tables
3. **Chunk** with overlap, respecting structure
4. **Embed** chunks in batches
5. **Store** vectors + text + metadata in a vector DB

Query pipeline (online)

1. **Transform** query (rewrite / expand / decompose)
2. **Retrieve** via hybrid (vector + BM25 keyword)
3. **Rerank** with a cross-encoder
4. **Assemble** context + cite sources
5. **Generate** a grounded answer; log everything

Chunking — where most quality is won or lost

- **Size:** 256–512 tokens works for most corpora; add **10–20% overlap** so answers aren't split mid-thought.
- **Respect structure:** never split tables mid-row, code mid-function, or sentences mid-clause — the classic "technically relevant but useless" chunk.
- **Attach metadata** (source, title, date, section) to every chunk for filtering & citations.
- **Semantic / recursive splitting** beats fixed-size for messy real-world docs.

Retrieval quality multipliers

Technique	Why it works	Typical impact
Hybrid search (dense + BM25)	Catches both semantic matches and exact keywords/IDs	~72% of prod systems use it; +1–9% recall
Cross-encoder reranking	Re-scores top-k for true relevance ordering	The "10x quality multiplier"
Query rewriting	Fixes the vocabulary gap between question & docs	Big win on multi-turn chat
Semantic caching	Reuse answers for near-duplicate queries	Up to ~68% inference cost cut

DECISION RULE (MEMORIZE FOR INTERVIEWS)

Prompt engineering first (cheapest). **RAG** for fresh/changing knowledge and facts. **Fine-tuning** only for behavior/format/style that prompting can't hit — **not** for facts.

08. Vector Databases + a Real RAG Pipeline

Choosing a vector database (2026)

Database	p50 latency @1M	Deployment	Best for
Qdrant	~6 ms	Self-host / cloud (Apache-2.0)	Lowest latency, flexible hosting, native hybrid
Pinecone	~8 ms	Fully managed (serverless)	Zero ops; teams without infra engineers
Weaviate	~12 ms	Self-host / cloud	GraphQL API, built-in hybrid search
Chroma	~18 ms	Python package	Prototyping speed (migrate above ~5M vectors)
pgvector	varies	Postgres extension	Already on Postgres; keep one system

A production-shaped RAG pipeline (LangChain + Chroma)

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma

# --- INDEX (offline) ---
docs   = PyPDFLoader("handbook.pdf").load()
splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=80)
chunks = splitter.split_documents(docs)               # structure-aware chunks
store  = Chroma.from_documents(chunks, OpenAIEmbeddings(),
                               persist_directory="./db")

# --- QUERY (online) ---
retriever = store.as_retriever(search_kwargs={"k": 5})
hits = retriever.invoke("What is the refund window?")

if not hits:                                          # edge case: no context
    answer = "I don't have enough information to answer that."
else:
    context = "\n\n".join(f"[src:{d.metadata['source']}] {d.page_content}" for d in hits)
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    answer = llm.invoke(
        f"Answer ONLY from the context. Cite sources.\n{context}\n\nQ: refund window?").content
```

THE 3 FEATURES THAT BEAT TUTORIALS

1. Source attribution (cite the page). **2.** A relevance filter on retrieved chunks. **3.** Graceful "I don't know" instead of hallucinating when no chunk is relevant.

THE TRAP REVIEWERS SPOT IN 5 SECONDS

A chatbot UI over a tutorial corpus (Wikipedia, the Constitution) with **no eval set**. Build over a messy, real corpus you understand, and show accuracy with vs without each component.

09. Evaluation: the Skill That Gets You Hired

The single biggest differentiator between offers and rejections in 2026 interviews is **evals**. Candidates who can build a RAG pipeline but can't **measure** it get filtered out. RAG fails silently — it returns fluent, confident, **wrong** answers with no error thrown. Evaluation is the only instrument that can see it. Classic metrics (BLEU/ROUGE) are useless here; they measure n-gram overlap, not grounding.

The four canonical RAGAS metrics (know these cold)

Metric	Question it answers	Failure surface	Target
Faithfulness	Does the answer only make claims supported by the retrieved context?	Generation (hallucination)	> 0.85 (> 0.95 high-stakes)
Answer Relevancy	Does the answer actually address the question asked?	Often retrieval	> 0.80
Context Precision	Of chunks retrieved, how many were actually relevant?	Retrieval (noise)	> 0.70
Context Recall	Did retrieval fetch everything needed to answer?	Retrieval (misses)	> 0.80

THE DIAGNOSTIC THAT IMPRESSES INTERVIEWERS

Always **separate retrieval quality from generation quality.** "The retriever found the right doc but the answer is still wrong" → a **generation/faithfulness** problem. "The answer is off-topic" → usually a **retrieval** problem. Mixing these is the #1 reason teams can't improve their RAG systems.

Wire evals into CI — gate deploys on regressions

```python
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall

# 'dataset' = your golden set of 50-200 representative Q&A + expected sources
report = evaluate(dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall])

assert report["faithfulness"] > 0.85, "Regression: hallucination up, block deploy"
```

The evaluation workflow

1. **Build a golden set** of 50–200 real queries with expected answers/sources.
2. **Score offline** with RAGAS (exploration) → DeepEval (CI/CD) → monitor in production.
3. **Track over time**, slice by query type, and **gate deploys** on regressions.
4. **Treat user corrections** as signals to improve the retrieval index — eval is continuous.

Caveat interviewers want to hear: LLM-as-judge is powerful but biased and costs ~\$0.001–0.003/case — keep humans in the loop for high-stakes domains.

10. AI Agents & Frameworks

An agent is an LLM given an environment where it can loop: reason, call tools, observe results, and continue until a goal is met. RAG **retrieves**; tool calling **acts**; agents **orchestrate** both over multiple steps. The framework you pick determines how state is persisted, how retries work, and how debuggable the system stays in production.

Framework	Sweet spot	Pick it when...
LangGraph	Stateful graph workflows; most battle-tested	Real structure: branches, retries, human-in-the-loop
CrewAI	Role-based multi-agent teams	You want a fast, intuitive multi-agent prototype
LlamaIndex Workflows	Event-driven, document/RAG-heavy	Data-intensive retrieval pipelines
Microsoft Agent Framework	Enterprise .NET/Azure (SK + AutoGen merged)	You're on the Microsoft stack
OpenAI Agents SDK / Pydantic AI	Tightly-scoped, type-safe Python agents	Minimal abstraction, clean delegation

All major frameworks now ship native MCP support. Choose by your dominant constraint (control, speed, language, state) — not by GitHub stars. It's common to combine them.

11. Model Context Protocol (MCP)

MCP is the "USB-C for AI." Introduced by Anthropic in late 2024 and now supported by every major vendor, it turns the messy **N×M** integration problem (every model × every tool) into a clean **N+M** one. Build one MCP **server** for a data source and **any** MCP-compatible client can use it.

Three roles

- **Host** — the AI app (Claude Desktop, Cursor, your agent)
- **Client** — connector inside the host managing sessions
- **Server** — exposes capabilities (GitHub, Postgres, Slack...)

Three primitives

- **Tools** — functions the model can invoke
- **Resources** — read-only data/context
- **Prompts** — reusable templated workflows

```python
# A minimal MCP server with FastMCP
from fastmcp import FastMCP
mcp = FastMCP("my-service")

@mcp.tool()
def search_orders(query: str, limit: int = 10) -> list[dict]:
    """Search orders by keyword."""
    return db.search(query, limit=limit)
```

INTERVIEW-READY FRAMING

MCP and function calling are **not competitors**: function calling is the model API; MCP is the integration layer above it. It runs on JSON-RPC 2.0 over stdio (local) or HTTP/SSE (remote). **Security lives in the host** — consent, credential scope, and the server allowlist are yours to design.

12. Shipping to Production: FastAPI + Docker

The gap between a demo and a hireable engineer is deployment. Most LLM demos die in a notebook; production needs async handling, streaming, health checks, secrets discipline, and monitoring. This is where your existing SWE skills shine.

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
app = FastAPI(title="AI Service")

@app.get("/health")                    # readiness gate for orchestrators
async def health(): return {"status":"ok"}

@app.post("/chat/stream")              # stream = better perceived latency
async def chat(prompt: str):
    def gen():
        for chunk in llm_stream(prompt): yield chunk
    return StreamingResponse(gen(), media_type="text/plain")
```

- **Base image:** `python:3.11-slim` + multi-stage build (cuts PyTorch images 60–80%).
- **Secrets** via env vars injected at runtime — never baked into the image or committed.
- **Pre-warm** model loading in a background thread so the first request doesn't time out.
- **Monitor** per-request tokens, latency, cost, model, and retry count — you can't cut what you can't see.

13. Cost & Latency Optimization

Most production LLM bills are **5–10× larger than necessary.** Output tokens cost 3–10× more than input tokens — so response-shape discipline is high-leverage. Layer these levers:

Lever	How	Typical saving
Prompt caching	Cache the large static system prompt/context	60–80% on chat/agents
Model routing	Cheap model for easy tasks, flagship only for hard ones	50–74%
Semantic caching	Reuse answers for near-duplicate queries (cosine > 0.95)	30–60% (repetitive)
Batch API	For async / offline pipelines	~50%
Output-shape control	"Be concise / respond in JSON" trims tokens	20–40%

RULE

"Use a cheaper model" is **one** lever of seven — the last you reach for, not the first. Instrument cost per request first, then pull the highest-leverage lever for your workload.

14. Guardrails & Safety

Every LLM shipped without guardrails is a liability — it **will** eventually leak data, get jailbroken, or hallucinate convincingly. It's a question of **when**, not if. Guardrails are runtime controls that live **outside** the model, inspecting inputs before the model sees them and outputs before users do. Prompt injection is #1 on the OWASP LLM Top 10.

The five risk categories every stack must cover

Risk	What it looks like	Where to defend
Prompt injection	"Ignore previous instructions..."; malicious text hidden in RAG docs or tool outputs	Input scanning + context segmentation + output checks
PII leakage	Model echoes SSNs, emails, keys	PII detection/anonymization on in & out
Topic drift	Steered into legal/medical/off-brand advice	Topical rails / allow-list
Hallucination	Invented facts, URLs, citations	RAG grounding + verification pass
Toxic output	Offensive/biased content	Moderation API backstop

The layered stack (2026)

- Fast scanner (LLM Guard) for cheap pre-filtering
- Programmable rails (NeMo Guardrails) for flow control
- Structured-output validator (Guardrails AI)
- Provider moderation (OpenAI/Anthropic) as backstop

Adds ~50–200ms. No single tool covers all five risks.

Agentic systems break hardest

When an agent can update records or trigger refunds, "safety" means constraining **actions**, not just text. Add capability scoping, sandboxing, tool-call authorization gates, and an audit trail. Guardrails are a seat-belt, not armor — red-team them continuously.

SAY THIS IN INTERVIEWS

"Guardrails are runtime, application-layer controls — distinct from training-time alignment (RLHF, Constitutional AI). For a write-capable agent, the model **proposing** a tool call must never mean the tool **executes**; I gate consequential actions behind validation." That one sentence signals senior judgment.

15. The 90-Day Migration Plan

A tight, focused sprint — not a year of tutorials. Three phases of 30 days. Realistic outcome for a working software engineer: **first competitive interview at ~3 months, offer at 4–6.**

Days 1–30
Learn the LLM stack, hands-on

Days 31–60
Ship ONE real project

Days 61–90
Target & interview

Phase 1 · Days 1–30 — The LLM Stack

- ✓ Call LLM APIs; master the Role+Context+Task+Format prompt structure.
- ✓ Structured outputs with Pydantic; few-shot & chain-of-thought.
- ✓ Build a basic RAG over your own documents (chunk → embed → retrieve).
- ✓ Write your first eval; add tool/function calling to a small script.

Rule: build a tiny thing every day. Not tutorials — reps.

Phase 2 · Days 31–60 — Ship One Real Thing

- ✓ Pick a real problem in a domain you understand; deploy it live (real users, even 10).
- ✓ Add hybrid retrieval + reranking + citations + graceful "I don't know."
- ✓ Wrap it in FastAPI, containerize with Docker, add a /health check.
- ✓ Build a golden eval set; show accuracy with vs without each component.

Anyone can follow a tutorial. Almost nobody ships. Shipping is the leap.

Phase 3 · Days 61–90 — Target & Interview

- ✓ Target companies that hire transitioners; apply broadly to AI-product roles.
- ✓ Practice the RAG system-design question out loud (Section 17).
- ✓ Reframe your resume: production experience is the headline, AI the new layer.
- ✓ Tell your migration story — you bring rare, in-demand skills to a starved market.

16. Portfolio Projects That Get You Hired

Hiring managers spend ~90 seconds on your portfolio, scanning for **production signals** — error handling, eval rigor, observability — not project count. Build **three deeply**, not eight shallowly. Recruiters now ask for **links** before resumes.

1. Production-grade RAG over a real corpus must-build

Over a messy domain you know (your team's handbook, a podcast archive). Include hybrid retrieval (dense + BM25), cross-encoder reranking, query rewriting, and trustworthy citations. Show a held-out eval set with accuracy **with vs without** each component.

Stack: Python + FastAPI, OpenAI/Claude, Qdrant/pgvector, a reranker, LangChain/LlamaIndex, Docker.

2. A multi-step agent that solves a real workflow must-build

Tool calling + planning + memory, with a clear trust boundary, argument validation, and an audit trail. Name the failure modes and how you handle them (interviewers love this).

3. Pick ONE based on your target role

- **Applied-AI:** an evals-first project / LLM evaluation framework (RAGAS).
- **Infra:** an LLM observability + cost dashboard (tokens, latency, drift).
- **Structured data:** a reliable LLM extraction pipeline with Pydantic + FastAPI.

WHAT MAKES A PROJECT "HIREABLE"

A clear problem statement, an **end-to-end deployed** solution, **measured results** (accuracy, latency, cost), and a readable README that documents **what failed, what you tried, what you learned.** Documenting failure is a green flag, not a weakness — it signals real engineering judgment.

17. Interview Prep: System Design + Real Q&A

2026 AI-engineer loops are roughly **40% RAG/evals/agents, 30% production systems, 20% LLM internals, 10% behavioral.** A classic coding screen often remains. Every round grades one instinct: do you treat an LLM feature as a **production system** (cost, latency, evaluation, failure modes) — not a clever prompt?

The 6-step framework for any AI system-design question

1. **Clarify** functional + non-functional requirements and true SLOs (spend the first 5 minutes here).
2. **Capacity estimate** — QPS, tokens/request, storage, GPU economics.
3. **High-level architecture** — boxes and arrows.
4. **Deep-dive** key components — models, indices, data flow.
5. **Failure modes & trade-offs** — what breaks at scale, and cost.
6. **Evaluation & observability** — how you **know** it works.

The #1 reason strong engineers fail: jumping straight to a Kafka-and-microservices diagram before establishing what the system must actually do.

High-signal questions — weak vs strong answers

Question	Weak	Strong
RAG vs fine-tuning?	"Fine-tuning is more accurate."	"RAG for fresh/changing facts; fine-tuning for format/behavior, not facts. Start with RAG."
Retriever finds the right doc but the answer is wrong — isolate it?	"Tweak the prompt."	"Separate retrieval from generation: confirm the passage reached the model, then measure faithfulness vs relevancy on a labeled set."
Reduce hallucinations?	"Lower temperature."	"Ground with RAG, require citations, constrain with structured output, add a verification pass — and measure faithfulness. Can't reduce what you don't measure."
Agent that updates an account — what's allowed without confirmation?	"Give it all the tools."	"Define the trust boundary first: separate reads from writes, validate args, use idempotency, audit trail, human gate on consequential writes."
Control latency & cost?	"Use a smaller model."	"Route by task, cache the system prompt, stream for perceived latency, batch async work, trim output tokens — instrument first."

THE DIFFERENTIATOR

The agents round is where people fail — they can describe ReAct but can't name its failure modes. Come armed with: wrong-tool selection, bad arguments, infinite loops, and unbounded cost — and your mitigation for each.

PART IV · SECTION 18

18. Salary, the Window & Your 7-Day Kit

The salary reality (2026)

Level	US base	US total comp	India (experienced)
Junior (0–2 yr)	\$120–170K	up to ~\$175K	~₹40–95 LPA *(role & company dependent)*
Mid (2–4 yr)	\$170–240K	\$230–380K	
Senior (4–7 yr)	\$220–310K	\$340–550K	
Staff / Lead	\$280–400K	\$500–800K	

"AI Engineer" has overtaken "ML Engineer" as the highest-paid software specialization. Median AI-engineering salary sits around **\$176K**; agents, LLM, and cloud are the highest-demand skills.

THE WINDOW IS NARROWING

In 2023 a notebook and enthusiasm got you hired. In 2026 the bar has risen (real RAG system design, eval deep-dives) because candidate supply is catching up. It is still the strongest hiring window since 2021 — for people who **ship**. The engineers who move now walk through comfortably; those who wait find it harder each quarter.

Your 7-day starter kit — start today, not Monday

- ✓ **Day 1:** Get an API key.
- ✓ **Day 2:** Make your first LLM call.
- ✓ **Day 3:** Build a tiny RAG over one PDF.
- ✓ **Day 4:** Add a simple eval (faithfulness).
- ✓ **Day 5:** Deploy it somewhere public.
- ✓ **Day 6:** Write the README (incl. failures).
- ✓ **Day 7:** Share it publicly — get feedback.
- ✓ **Then:** repeat the loop — Learn → Build → Ship → Reflect.

Traps that quietly stall you

- ⚠ Tutorial hell — endlessly watching, never building.
- ⚠ Chasing every shiny new tool instead of going deep on fundamentals.
- ⚠ Studying theory forever because building feels scary. Don't let comfort disguise itself as progress.

RECAP · THE MIGRATION MINDSET

You're not starting over. You're upgrading a system that works.

You're 70% there

Your shipping discipline, APIs, testing & reliability instincts already transfer.

Learn Tier 2

LLM APIs, RAG, evals, agents — not the academic Tier 3 rabbit holes.

Ship one real thing

A deployed, evaluated project beats any certificate on earth.

Keep going — on the channel

This playbook is the companion to the full video breakdown on **TechWithKoushik**. If it helped, the best thank-you is simple:

SUBSCRIBE **LIKE** **COMMENT your Phase 1 start date**

TechWithKoushik

More free guides, cheat sheets & roadmaps:

koushikmaji.com

Now go make your first LLM call.
Your migration starts **today.**